Modifying LeanCoP Options on the Fly

(aka "the PAAR/AITP paper")

Introduction

In LeanCoP, small changes to proof-search control have large practical effects [**ottenLeanCoP20**, **Otten2023leanCoP20**]. The current project studies such changes in a PyCoP-style environment where each proof step is an explicit action. This makes it possible to experiment with `cut`, `comp(J)`, and explicit `Reset(n)` actions under direct program control rather than only through static prover options.

Three lines of prior work motivate the current strategy set. First, **letzModelEliminationConnection2001** argue for choosing proof steps that should lead to fewer solutions, which suggests preferring reductions and shorter clauses. Second, **zomboriRoleEntropyGuiding2021** show that entropy is a useful proxy for search confidence; this suggests committing with cuts only when the local action distribution is sharp. Third, **farberCuriouslyEffectiveBacktracking2024** decomposes restricted backtracking into separate decisions about cuts on reductions and on extensions, and reports that reduction cut is often beneficial while extension-cut variants differ substantially in strength.

Can a lightweight hand-written controller can outperform the standard LeanCoP-style `cut + comp(7)` baseline on a meaningful fragment of TPTP? No single replacement strategy achieves this yet, but the experiments show substantial complementarity across strategies — enough to be useful for schedules and learned controllers.

Strategies Used in the Schedules

The current schedule experiments use the following strategies and no others. They vary along a small number of interpretable axes: forward-action ordering, how aggressively `cut` is used, and whether search may react to dead ends by resetting the depth bound. That makes the resulting portfolios easier to analyse than a collection of unrelated tweaks.

FirstAction
: Run the first legal action. This is the PyCoP default depth-first baseline.

LeanCopCut
: Mimic leanCoP's inclusive restricted backtracking: cut successful reductions and extensions, and use `comp(7)` to recover completeness [**ottenLeanCoP20**].

ReductionCut
: Apply inclusive cut only on reductions. This isolates the "R" component in Färber's decomposition [**farberCuriouslyEffectiveBacktracking2024**].

HeuristicReductionCut
: Choose forward actions heuristically, preferring reductions and then shorter clauses, and cut only on reductions.

SubgoalReordering
: Use a stricter categorical order: reductions first, then shorter extensions, then shorter starts [**letzModelEliminationConnection2001**].

EntropyCut
: Follow the baseline order, but annotate a forward action with `cut` only when the softmax entropy of heuristic action scores is low [**zomboriRoleEntropyGuiding2021**].

AdaptiveReset
: Replace some dead-end backtracks by explicit resets to a larger depth bound, based on consecutive failures and a simple search-state heuristic. This is inspired by restart control in SAT [**HoldenSATQSAT**, **liReinforcementLearningBased2024**].

SmartCut
: Combine entropy-based cut with adaptive reset.

FewAlternativesCut
: Keep the default depth-first order and leanCoP-style `comp(7)`, but only apply `cut` when the current choice point is already narrow.

GoalDistanceCut
: Keep leanCoP-style cut, but choose forward actions using an estimate of how close the resulting subgoals are to closure.

DepthPhasedCut
: Cut reductions throughout the search, but cut ex-

tensions only near the top of the current iterative-deepening tree, using `comp(10)` rather than `comp(7)`.

Controls and Schedules

The strategy complementarity visible in the individual runs makes static serial portfolios a natural next step. I therefore build four time-budgeted schedules, alongside the two single-strategy controls `FirstAction` and `LeanCopCut`. Together they give two simple controls and four optimised portfolios under two budgets and two objectives.

FirstAction
Depth-first control with no portfolio effect. It is included as a weak control for the value of any scheduling at all.

LeanCopCut
The main single-strategy control. It is both the strongest hand-written baseline and, for the "novel" objective, the reference against which new solves are counted.

P1 60s
60-second schedule chosen to maximise the total number of solved problems. Its slots are `AdaptiveReset`, `SubgoalReordering`, `LeanCopCut`, `SmartCut`, `FirstAction`, and `GoalDistanceCut`.

P2 60s
60-second schedule chosen to maximise solves not already obtained by `LeanCopCut`. Its slots are `SmartCut`, `EntropyCut`, `GoalDistanceCut`, `HeuristicReductionCut`, `AdaptiveReset`, `ReductionCut`, `SubgoalReordering`, and `FewAlternativesCut`.

P1 120s
120-second schedule chosen to maximise the total number of solved problems. Its slots are `SubgoalReordering`, `AdaptiveReset`, `LeanCopCut`, `HeuristicReductionCut`, `EntropyCut`, `GoalDistanceCut`, and `DepthPhasedCut`.

P2 120s
120-second schedule chosen to maximise solves not already obtained by `LeanCopCut`. Its slots are `SmartCut`, `SubgoalReordering`, `DepthPhasedCut`, `HeuristicReductionCut`, `GoalDistanceCut`, `EntropyCut`, `AdaptiveReset`, and `ReductionCut`.

The portfolio construction problem is a multiple-choice budgeted maximum-coverage problem. For each strategy and each admissible time cap, the pair "strategy + cap" covers exactly the benchmark problems that the strategy solves within that cap. The search then chooses at most one cap per strategy, subject to a total time budget, so as to maximise the size of the covered union. This first phase is solved exactly as a mixed-integer linear program.

This still leaves the order of the chosen schedule slots. For a fixed set of capped strategies, the second phase solves an exact serial-portfolio ordering problem: choose the permutation that minimises total benchmark runtime. I solve this by dynamic programming over subsets of the selected slots, using the recorded per-problem runtimes as stage costs and charging the full slot cap on timeout rows. In other words, the schedules are not greedy hand-tuned portfolios; they are exact optima for the chosen objective under the runtime model induced by the canonical experiments.

Results

The main quantitative comparison is shown in **??**. For each budget, I report the total number of solved problems, the number solved but not by `LeanCopCut`, the number solved but not by `FirstAction`, and the number solved by the schedule while remaining unsolved by both controls. The two portfolio objectives behave as intended: the P1 portfolios improve overall coverage, while the P2 portfolios recover more control-escaping problems.

The violin plots in **??** use exactly the same score as the violin plots in Mangla's thesis [**Mangla2025MLTPS**]. For a run with budget T and consumed time t, the score is

$$s = 0.5 + \frac{T - t}{2T} \text{ on success, and } s = \frac{T - t}{2T} \text{ otherwise.}$$

Thus all successful runs lie above 0.5, failures lie at or below 0.5, and a full-budget failure receives score 0. This makes the violin plots a compact summary of both coverage and proof speed.

Table 1: Solved-problem counts for the controls and the four portfolios.

Setting	Solved	Not by `LeanCopCut`	Not by `FirstAction`	Not by either
`FirstAction` (60s)	2049	174	0	0
`LeanCopCut` (60s)	2164	0	289	0
`P1 60s`	2291	171	323	64
`P2 60s`	2140	247	174	101
`FirstAction` (120s)	2134	166	0	0
`LeanCopCut` (120s)	2304	0	336	0
`P1 120s`	2449	173	387	74
`P2 120s`	2292	256	227	113

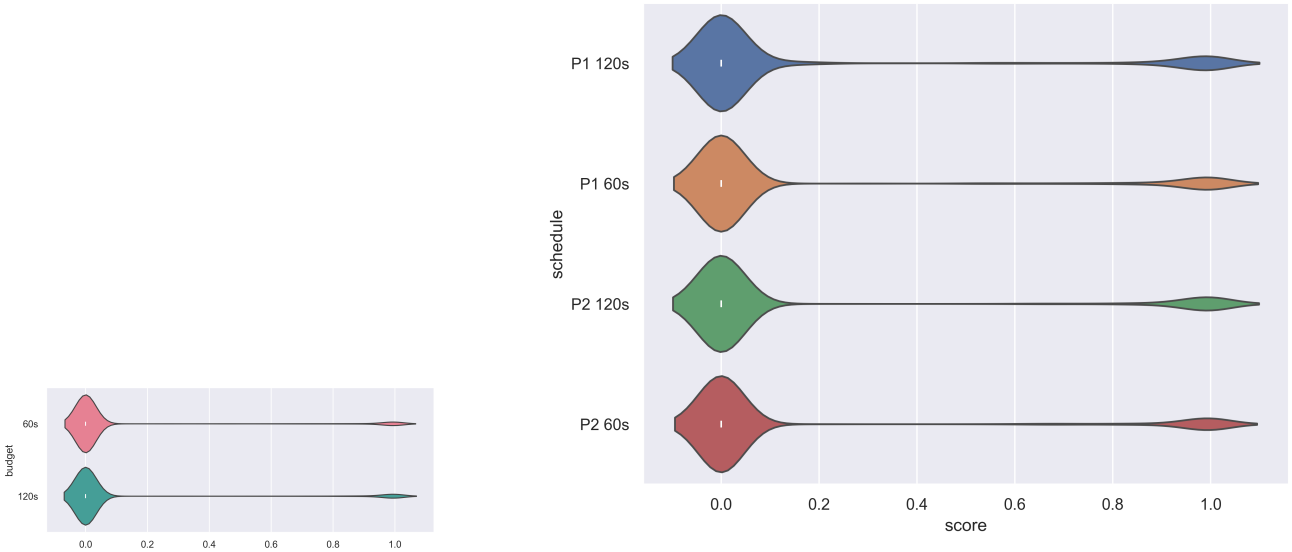


Figure 1: Violin plots of the Mangla-style score distributions for the controls (left) and the portfolios (right).

Discussion

Three observations stand out.

1. Reduction cut really does look like a meaningful design axis in its own right, not just a weakened version of `LeanCopCut`. That matches Färber's R/EI/EX decomposition [**farberCuriouslyEffectiveBacktracking2024**].

2. The present strategy family already supports sensible schedule and controller experiments, because the strategies differ along intelligible axes rather than being a bag of unrelated tweaks.

3. The largest remaining gap is still exclusive cut on extension steps (REX/EX). The current Boolean `cut` flag only supports inclusive cut, so the backtracking machinery would need to be extended to preserve root alternatives.

What remains to turn this into a paper:

1. evaluate portfolios and per-problem controllers that mix `LeanCopCut` with the most complementary strategies under fixed time budgets;

2. implement a genuine REX-style extension cut in the calculus and compare it directly with the inclusive variants [**farberCuriouslyEffectiveBacktracking2024**].